

AbstractFactory

1. Explicación del patrón

Abstract Factory es un patrón de creación cuyo objetivo es crear objetos agrupados en familias sin que el cliente tenga que conocer las clases concretas que se usan para crear esos objetos.

La idea principal es que el cliente no instancie directamente los productos concretos. En su lugar, utiliza una fábrica abstracta, que define métodos de creación para cada tipo de producto de la familia.

Después, cada fábrica concreta se encarga de crear productos concretos pertenecientes a una misma familia.

De esta forma, el sistema puede trabajar con familias completas de productos relacionados manteniendo bajo acoplamiento con las clases concretas.

Por ejemplo, si una aplicación necesita crear distintos tipos de vehículos, como automóviles y scooters, puede tener varias familias:

- Vehículos eléctricos
- Vehículos de gasolina

Cada familia tendrá sus propios productos concretos:

- Automóvil eléctrico
- Scooter eléctrico
- Automóvil gasolina
- Scooter gasolina

El cliente solo conoce la fábrica abstracta y los productos abstractos, no las clases concretas.

2. Cuándo se utiliza

Se utiliza el patrón Abstract Factory cuando:

- Un sistema necesita crear productos relacionados entre sí.
- Los productos se agrupan en familias.
- El cliente debe ser independiente de la forma en que se crean los productos.
- El cliente no debe depender de las clases concretas de los productos.
- Un sistema puede estar configurado con distintas familias de productos.
- Las familias de productos pueden evolucionar o cambiar.
- Se quiere garantizar que los productos creados pertenecen a la misma familia y son compatibles entre sí.
- Se quiere evitar crear objetos concretos directamente con `new` desde el código cliente.
- Se quiere centralizar la creación de objetos relacionados.

3. Cómo aplicarlo

Para aplicar Abstract Factory se suelen definir estos participantes:

Fábrica abstracta

Es una interfaz que declara los métodos de creación de los distintos productos.

Define qué productos se pueden crear, pero no cómo se crean ni qué clase concreta se instancia.

Ejemplo conceptual:

- `crearProductoA()`
- `crearProductoB()`

En el ejemplo de vehículos:

- `crearAutomovil()`
- `crearScooter()`

Fábricas concretas

Son las clases que implementan la fábrica abstracta.

Cada fábrica concreta representa una familia concreta de productos.

Responsabilidades:

- Implementar los métodos de creación definidos por la fábrica abstracta.
- Crear productos concretos de una misma familia.
- Garantizar que los productos creados son compatibles entre sí.

Ejemplo de fábricas concretas:

- `FabricaVehiculoElectricidad`
- `FabricaVehiculoGasolina`

La fábrica de vehículos eléctricos crea productos eléctricos.

La fábrica de vehículos de gasolina crea productos de gasolina.

Productos abstractos

Son las interfaces o clases abstractas que representan los tipos generales de productos.

No dependen de una familia concreta.

Ejemplo conceptual:

- `ProductoAbstractoA`
- `ProductoAbstractoB`

En el ejemplo de vehículos:

- `Automovil`

- Scooter

Productos concretos

Son las implementaciones reales de cada producto abstracto dentro de una familia concreta.

Ejemplos:

- AutomovilElectricidad
- AutomovilGasolina
- ScooterElectricidad
- ScooterGasolina

Cada producto concreto pertenece a una familia específica.

Cliente

Es la clase que utiliza la fábrica abstracta.

Responsabilidades:

- Trabajar con la interfaz de la fábrica abstracta.
- Pedir productos a la fábrica.
- Usar los productos a través de sus interfaces abstractas.
- No depender de las clases concretas de las fábricas ni de los productos.

El cliente recibe una fábrica concreta, pero la trata como fábrica abstracta.

4. Forma práctica de aplicarlo

Pasos:

1. Detectar qué objetos se crean en grupo o familia.
2. Identificar los tipos generales de productos que existen.
3. Crear una interfaz o clase abstracta para cada tipo de producto.
4. Crear las clases concretas de cada producto para cada familia.
5. Crear una interfaz `FabricaAbstracta` con un método de creación por cada producto abstracto.
6. Crear una fábrica concreta por cada familia de productos.
7. Hacer que cada fábrica concreta devuelva productos de su propia familia.
8. Modificar el cliente para que use solo la fábrica abstracta.
9. Sustituir las llamadas directas a constructores por llamadas a métodos de la fábrica.
10. Seleccionar la fábrica concreta adecuada al inicio de la aplicación o según la configuración.

5. Idea clave

El cliente no debe crear productos concretos directamente.

Debe pedirlos a una fábrica abstracta.

Así, el cliente puede trabajar con distintas familias de productos sin cambiar su código.

6. Ejemplo del material

Ejemplo de vehículos

En el material aparece un ejemplo con vehículos.

La fábrica abstracta es:

- `FabricaVehiculo`

Define métodos como:

- `crearAutomovil()`
- `crearScooter()`

Las fábricas concretas son:

- `FabricaVehiculoElectricidad`
- `FabricaVehiculoGasolina`

Los productos abstractos son:

- `Automovil`
- `Scooter`

Los productos concretos son:

- `AutomovilElectricidad`
- `AutomovilGasolina`
- `ScooterElectricidad`
- `ScooterGasolina`

La clase cliente, por ejemplo un catálogo, utiliza una instancia de `FabricaVehiculo`.

Si se le proporciona una fábrica eléctrica, creará automóviles y scooters eléctricos.

Si se le proporciona una fábrica de gasolina, creará automóviles y scooters de gasolina.

El cliente no necesita saber qué clase concreta se está instanciando.

7. Ejemplo conceptual de uso

Supongamos que una aplicación necesita crear productos de una familia concreta.

En vez de hacer esto:

```
new AutomovilElectricidad()
```

```
new ScooterElectricidad()
```

El cliente hace esto:

```
fabrica.crearAutomovil()
```

```
fabrica.crearScooter()
```

La fábrica concreta decide qué productos se crean.

Si la fábrica es eléctrica, crea productos eléctricos.

Si la fábrica es de gasolina, crea productos de gasolina.

8. Ventajas prácticas

- Evita que el cliente dependa de clases concretas.
- Agrupa la creación de productos relacionados.
- Garantiza que los productos creados pertenecen a la misma familia.
- Facilita cambiar una familia completa de productos.
- Permite añadir nuevas familias creando nuevas fábricas concretas.
- Centraliza la lógica de creación de objetos.
- Hace que el sistema sea más flexible ante cambios en las familias de productos.

9. Cuidado

Abstract Factory puede añadir muchas clases e interfaces.

Puede ser excesivo si solo se necesita crear un producto simple o si no existen familias de productos relacionadas.

También hay que tener en cuenta que añadir una nueva familia suele ser sencillo, porque basta con crear una nueva fábrica concreta y sus productos.

Sin embargo, añadir un nuevo tipo de producto puede requerir modificar la fábrica abstracta y todas las fábricas concretas existentes.

10. Diferencia práctica con Factory Method

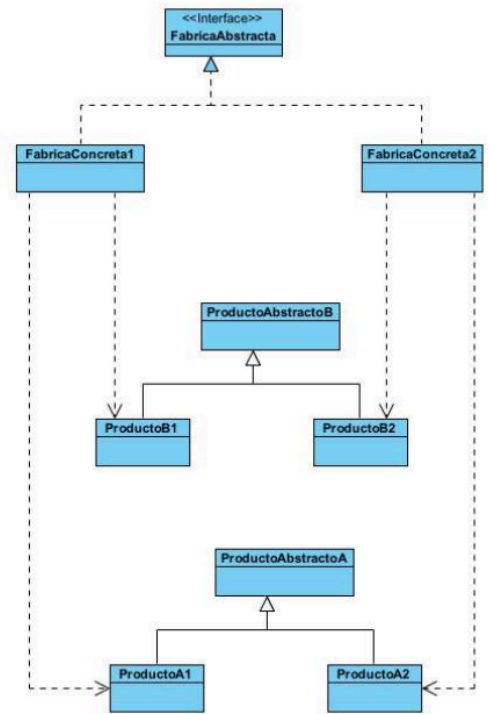
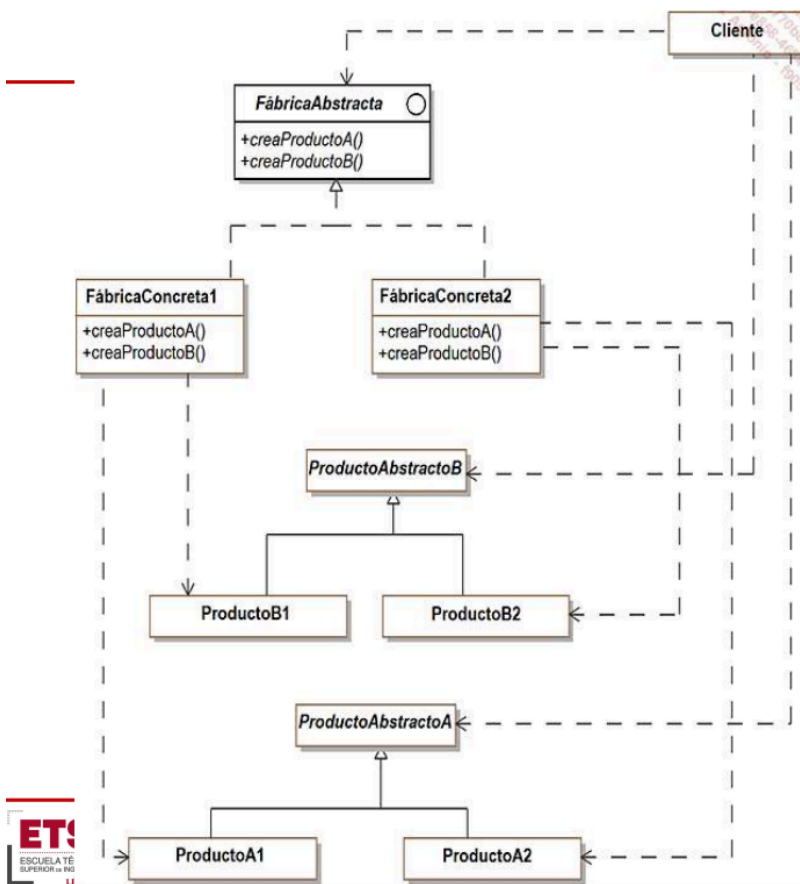
Factory Method se centra en delegar la creación de un producto concreto en una subclase o método de fábrica.

Abstract Factory se centra en crear familias completas de productos relacionados.

Por tanto:

- Factory Method suele crear un tipo de producto.
- Abstract Factory crea varios tipos de productos que pertenecen a una misma familia.

11. Plantilla



12. Ejemplo

Factor

